Rekursion – Konzepte, Strukturen und Umsetzung in funktionalen Programmiersprachen

aus dem Studiengang Wirtschaftsinformatik

an der Duale Hochschule Baden-Württemberg Mannheim

von

Emanuel Schuessler

Jannes Borgmann

08.05.2026

Matrikelnummer, Kurs: ???, WI25SEA
???, WI25SEA

Unternehmen: SAP SE, 69190, Walldorf
SAP SE, 69190, Walldorf

Betreuer an der DHBW: Edelbert Schumm

Inhaltsverzeichnis

1 Einleitung **1**

2 Definition und Grundlagen **2**
2.1 Was ist Rekursion? 3
2.2 Historischer Hintergrund 3
2.3 Strukturelle Rekursion 4

3 Rekursive Funktionen **4**
3.1 Das Fakultätsbeispiel 5
3.2 Terminierung 5
3.3 Endrekursion 6
3.4 Weitere Varianten 6

4 Rekursive Datenstrukturen **6**

5 Rekursion in der funktionalen Programmierung **7**
5.1 Rekursion statt Schleifen 8
5.2 Lazy Evaluation und unendliche Listen 8
5.3 Verbindung zum Lambda-Kalkül 8
5.4 Höherwertige Funktionen 9

6 Fazit **9**

Literatur ..... **a**

Glossar

Basisfall Der Abbruchfall einer rekursiven Funktion, der ohne weiteren Selbstaufruf einen Wert zurückgibt und so die Terminierung der Rekursion sicherstellt.

Endrekursion Eine spezielle Form der Rekursion, bei der der rekursive Aufruf die letzte Operation der Funktion ist. Viele Compiler können Endrekursion in eine Schleife umwandeln (Tail Call Optimization).

Pattern_Matching Ein Sprachkonstrukt in funktionalen Programmiersprachen, das es erlaubt, Werte anhand ihrer Struktur zu unterscheiden und zu destrukturieren.

Rekursion Ein Konzept, bei dem sich eine Funktion oder Datenstruktur direkt oder indirekt auf sich selbst bezieht. Eine rekursive Definition besteht stets aus einem Basisfall (Abbruchbedingung) und einem allgemeinen Fall (Selbstaufruf).

Selbstaufruf Der Teil einer rekursiven Funktion, in dem die Funktion sich selbst mit einem vereinfachten Argument aufruft.

Strukturelle_Rekursion Eine Form der Rekursion, die der Struktur des Eingabedatentyps folgt, z.B. bei der Verarbeitung von Listen oder Bäumen.

1 Einleitung

Rekursion begegnet uns häufiger, als man zunächst denkt. Ob das Blätterwerk eines Baumes, verschachtelte Ordnerstrukturen auf dem Computer oder die Grammatik einer Programmiersprache – viele Dinge sind von Natur aus rekursiv aufgebaut. In der Mathematik kennt man das schon lange: Die Fakultät, Folgen oder der euklidische Algorithmus basieren alle auf demselben Prinzip, auch wenn es dort selten so genannt wird.

In der funktionalen Programmierung ist Rekursion noch zentraler: Da Sprachen wie Haskell keine veränderbaren Variablen kennen, gibt es auch keine klassischen Schleifen. Stattdessen werden Wiederholungen immer durch Rekursion ausgedrückt [1]. Diese Ausarbeitung erklärt, was Rekursion ist, wie sie funktioniert, welche Varianten es gibt und wie sie in der Praxis eingesetzt wird.

2 Definition und Grundlagen

2.1 Was ist Rekursion?

Rekursion bedeutet, dass sich eine Funktion oder eine Datenstruktur in ihrer eigenen Definition auf sich selbst bezieht. Das klingt erst mal widersprüchlich, funktioniert aber, weil bei jedem Selbstaufruf das Problem ein Stück kleiner wird. Lorenz beschreibt es so: *„Man definiert etwas nicht explizit, sondern durch einfachere Versionen seiner selbst„* [2].

Damit das nicht ewig weitergeht, braucht jede Rekursion zwei Teile:

- **Basisfall** (auch Verankerung): Der einfachste Fall, der direkt beantwortet werden kann – ohne weiteren Selbstaufruf. Er sorgt dafür, dass die Rekursion irgendwann aufhört.
- **Allgemeiner Fall** (auch Vererbung): Das aktuelle Problem wird auf ein kleineres Problem desselben Typs zurückgeführt, indem die Funktion sich selbst mit einem verkleinerten Argument aufruft.

2.2 Historischer Hintergrund

Den ältesten bekannten rekursiven Algorithmus hat Euklid im dritten Jahrhundert vor Christus beschrieben: das Verfahren der wechselnden Wegnahme zur Berechnung des größten gemeinsamen Teilers. Die Idee: Zwei Zahlen a und b haben denselben größten gemeinsamen Teiler wie b und der Rest der Division a durch b. Man wiederholt das so lange, bis der Rest Null ist – dann ist man fertig [2].

Im 19. Jahrhundert formalisierte Giuseppe Peano die natürlichen Zahlen durch seine Peano-Axiome. Darin sind die natürlichen Zahlen induktiv definiert: Die Null existiert, und jede weitere natürliche Zahl ist der Nachfolger einer vorherigen. Grundoperationen wie Addition und Multiplikation lassen sich darauf aufbauend rekursiv definieren [2].

Im 20. Jahrhundert untersuchten Kurt Gödel und Alonzo Church, welche Funktionen überhaupt berechenbar sind. Gödel arbeitete dabei mit dem Konzept der primitiv-rekursiven Funktionen. In der Informatik wurde Rekursion dann mit der Sprache LISP (1958, John

McCarthy) zum praktischen Werkzeug: McCarthy erkannte, dass man viele Berechnungen direkt als rekursive Funktionen schreiben kann.

2.3 Strukturelle Rekursion

Besonders wichtig ist die **strukturelle Rekursion**: Eine Funktion folgt dabei dem Aufbau des Datentyps, den sie verarbeitet. Wenn ein Datentyp selbst rekursiv definiert ist, ergibt sich die passende Funktion darüber fast automatisch [2].

Das zeigt sich gut an den natürlichen Zahlen: Man kann sie induktiv beschreiben – entweder ist eine Zahl die Null (Basisfall), oder sie ist der Nachfolger einer anderen natürlichen Zahl (allgemeiner Fall). Eine Funktion über natürliche Zahlen behandelt genau diese zwei Fälle:

	Natürliche Zahlen	**Rekursive Funktion f(n)**
Basisfall	Die Zahl 0 existiert.	Für $n = 0$ gibt es eine direkte Antwort, z.B. $f(0) = 1$ bei der Fakultät.
Allg. Fall	Jedes $n > 0$ ist der Nachfolger von $n - 1$.	Für $n > 0$ wird $f(n-1)$ berechnet und mit n verknüpft: $f(n) = n \cdot f(n-1)$.

Tabelle 1 — Strukturelle Entsprechung zwischen Datentyp und Funktion

Das Prinzip: Wer den Aufbau des Datentyps kennt, weiß bereits, welche Fälle die Funktion behandeln muss.

3 Rekursive Funktionen

3.1 Das Fakultätsbeispiel

Die Fakultät ist das klassische Beispiel. Sie ist definiert als das Produkt aller ganzen Zahlen von 1 bis n, also z.B. $4! = 1 \cdot 2 \cdot 3 \cdot 4 = 24$. Rekursiv lässt sich das so ausdrücken: Die Fakultät von 0 ist 1 (Basisfall), und für alle anderen n gilt $n! = n \cdot (n-1)!$ (allgemeiner Fall).

In Java schreibt sich das fast genauso:

```java
public static long factorial(int n) {
    if (n == 0) {
        return 1;                   // Basisfall
    }
    return n * factorial(n - 1); // allgemeiner Fall
}
```

Listing 1 — Rekursive Berechnung der Fakultät in Java

Was passiert beim Aufruf `factorial(4)`? Die Methode ruft sich selbst immer wieder auf, bis sie bei `factorial(0)` ankommt und direkt 1 zurückgibt. Danach werden die Ergebnisse rückwärts zusammengesetzt: 1, dann $1 \cdot 1 = 1$, dann $2 \cdot 1 = 2$, dann $3 \cdot 2 = 6$, schließlich $4 \cdot 6 = 24$.

3.2 Terminierung

Eine Rekursion muss irgendwann aufhören. Das funktioniert nur, wenn das Argument bei jedem Aufruf strikt kleiner wird und irgendwann den Basisfall erreicht [2]. Bei der Fakultät nimmt n bei jedem Schritt um genau 1 ab – das führt zwingend zu $n = 0$.

Fehlt der Basisfall oder wird das Argument nicht verkleinert, läuft die Rekursion endlos. In Java endet das mit einem `StackOverflowError`, weil für jeden offenen Aufruf Speicher auf dem Aufrufstapel belegt wird – und der ist nicht unbegrenzt groß.

3.3 Endrekursion

Genau dieser Speicherverbrauch ist ein praktisches Problem bei tiefer Rekursion. Bei der normalen Fakultät muss Java sich für jeden offenen Aufruf merken, mit welchem n es noch multiplizieren muss. Das ergibt bei $n = 10.000$ auch 10.000 offene Einträge auf dem Stack.

Die **Endrekursion** (englisch: tail recursion) löst das: Eine Funktion ist endrekursiv, wenn der rekursive Aufruf die allerletzte Operation ist und das Ergebnis direkt zurückgegeben wird – ohne dass danach noch etwas damit passiert. Der Compiler kann dann den alten Stackeintrag einfach wiederverwenden. Dafür führt man einen Akkumulator ein, der das Zwischenergebnis mitträgt:

Statt `return n * factorial(n-1)` (Multiplikation nach dem Aufruf) schreibt man den rekursiven Aufruf so, dass die Multiplikation bereits vorher erledigt und als Parameter übergeben wird. In Sprachen wie Haskell ist diese Optimierung garantiert. In Java hingegen optimiert die Standard-Laufzeitumgebung Endrekursion nicht automatisch – dort ist man besser mit einer Schleife bedient [2].

3.4 Weitere Varianten

Bei der **gegenseitigen Rekursion** rufen sich zwei Funktionen abwechselnd auf. Zum Beispiel: `istGerade(n)` ruft `istUngerade(n-1)` auf, und umgekehrt. Beide bilden zusammen eine Rekursion, bis der Basisfall ($n = 0$) erreicht ist.

Bei der **verschachtelten Rekursion** ist der Selbstaufruf selbst wieder ein Argument eines weiteren Selbstaufrufs. Das bekannteste Beispiel ist die **Ackermann-Funktion**. Sie ist für alle natürlichen Zahlen berechenbar und terminiert immer – aber sie wächst so extrem schnell, dass sie schon für kleine Eingaben astronomische Werte annimmt. Wichtig: Die Ackermann-Funktion ist **nicht** primitiv-rekursiv, also nicht durch einfache Schleifen mit vorher bekannter Schrittzahl darstellbar. Das zeigt, dass Rekursion ausdrucksstärker ist als Schleifen [2].

4 Rekursive Datenstrukturen

Nicht nur Funktionen, sondern auch Datenstrukturen können rekursiv aufgebaut sein. Man nennt sie **induktive Datentypen**: Ihre Definition besteht aus einem Basisfall und einem allgemeinen Fall, der auf den Typ selbst verweist [3].

Listen sind das bekannteste Beispiel. Eine Liste ist entweder leer (Basisfall) oder sie hat ein erstes Element (Kopf) und dahinter eine weitere Liste (Schwanz). Funktionen über Listen folgen genau diesem Aufbau: Der Basisfall behandelt die leere Liste direkt, der allgemeine Fall verarbeitet den Kopf und ruft sich rekursiv für den Schwanz auf. Die Länge einer leeren Liste ist 0, sonst 1 plus die Länge des Schwanzes – ganz einfach.

Bäume funktionieren genauso. Ein Binärbaum ist entweder ein leeres Blatt (Basisfall) oder ein Knoten mit einem Wert und zwei Teilbäumen (allgemeiner Fall). Die Tiefe eines Baumes berechnet man rekursiv: Man fragt die Tiefen des linken und rechten Teilbaums ab und nimmt das Maximum. Ein leeres Blatt hat Tiefe 0.

Der Vorteil: Wenn man den Aufbau der Datenstruktur kennt, ergibt sich die Struktur der passenden Funktion von selbst. Das macht rekursive Programme über solche Typen leichter zu schreiben und zu verstehen [3].

5 Rekursion in der funktionalen Programmierung

5.1 Rekursion statt Schleifen

In Sprachen wie Java benutzt man `for`- oder `while`-Schleifen für Wiederholungen – dabei wird eine Variable mit jedem Durchlauf verändert. In rein funktionalen Sprachen wie Haskell gibt es das nicht: Einmal zugewiesene Werte bleiben unveränderlich [1].

Deshalb übernimmt dort Rekursion die Rolle der Schleifen. Statt eine Variable zu erhöhen, übergibt man den neuen Wert als Argument an den nächsten Aufruf. Das Ergebnis ist dasselbe, nur ohne veränderbaren Zustand. Durch Endrekursion und die dazugehörige Compileroptimierung ist das auch in Bezug auf Speicher und Geschwindigkeit gleichwertig zu einer Schleife.

5.2 Lazy Evaluation und unendliche Listen

Haskell wertet Ausdrücke erst aus, wenn ihr Wert wirklich gebraucht wird – das nennt sich **lazy evaluation** (bedarfsgesteuerte Auswertung). Kombiniert mit Rekursion kann man damit sogar unendliche Datenstrukturen definieren, z.B. eine Liste aller natürlichen Zahlen. Haskell berechnet dann nur so viele Elemente, wie tatsächlich benötigt werden. Solche potenziell unendlichen Listen nennt man auch **Streams** und sie sind eng mit dem Thema Listen verbunden [4].

5.3 Verbindung zum Lambda-Kalkül

Im Lambda-Kalkül – dem theoretischen Fundament der funktionalen Programmierung – gibt es keine Möglichkeit, Funktionen direkt zu benennen und sich selbst aufzurufen. Rekursion wird dort über den **Y-Kombinator** umgesetzt, einen sogenannten Fixpunktoperator. Er gibt für eine Funktion f denjenigen Wert x zurück, für den gilt $f(x) = x$. Damit kann man Rekursion aus dem Lambda-Kalkül heraus ableiten, ohne sie eingebaut zu haben [1].

5.4 Höherwertige Funktionen

In der Praxis schreibt man in Haskell Rekursion oft nicht direkt aus, sondern benutzt höherwertige Funktionen wie `map`, `filter` oder `foldr`. Diese kapseln häufig vorkommende Rekursionsmuster. `foldr` zum Beispiel verallgemeinert die strukturelle Rekursion über Listen: Es kombiniert alle Elemente einer Liste durch eine übergebene Funktion von rechts nach links. Summe, Länge oder Umkehrung einer Liste lassen sich alle als Spezialfall von `foldr` schreiben. Die Rekursion steckt dabei immer noch drin – sie ist nur versteckt [3].

6 Fazit

Rekursion ist kein komplizierter Trick, sondern ein natürliches Denkprinzip: Man löst ein Problem, indem man es auf eine kleinere Version desselben Problems zurückführt. Das haben Menschen schon vor mehr als 2000 Jahren gemacht – mit dem euklidischen Algorithmus.

In der funktionalen Programmierung ist Rekursion besonders wichtig, weil es ohne veränderliche Variablen keine Schleifen geben kann. Wer versteht, wie induktive Datentypen aufgebaut sind, kann darüber fast automatisch korrekte rekursive Funktionen schreiben. Konzepte wie Endrekursion zeigen außerdem, dass rekursive Programme auch effizient sein können – nicht nur elegant.

Literatur

[1] P. Pepper, *Funktionale Programmierung in OPAL, ML, Haskell und Gofer*. Springer Verlag, 2006.

[2] G. Lorenz, *Funktionale Modellierung und Rekursion: Praxis – Didaktik – Theorie*. München: Oldenbourg Verlag, 2012.

[3] M. Block und A. Neumann, *Haskell Intensivkurs*. Springer Verlag, 2011.

[4] V. Krypczyk und O. Bochkor, *Handbuch für Softwareentwickler*, 2. Aufl. Bonn: Rheinwerk Verlag, 2021.